



AUDIT REPORT

February 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	08
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Informational Issues	14
1. Lack of Input Validation	14
2. Floating Pragma	15
3. Use Two-Step Ownership Transfer Instead of Standard Ownable	16
Functional Tests	17
Automated Tests	17
Threat Model	18
Closing Summary & Disclaimer	24



Executive Summary

Project Name	PuffpawOFT
Protocol Type	OFT Bridge
Project URL	https://www.puffpaw.xyz/
Overview	Smart contracts implementing LayerZero OFT (Omnichain Fungible Token) standard for cross-chain token transfers. The assessment covers PPOFTAdapter (origin chain wrapper for existing ERC20 tokens) through MyOFT (destination chain native token contract), both utilizing LayerZero's messaging protocol for seamless omnichain fungibility and interoperability.
Audit Scope	The scope of this Audit was to analyze the Puffpaw OFT Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/puffpaw-lab/puffpaw-sol-bridge
Contracts in Scope	oft/contracts/MyOFT.sol oft/contracts/PPOFTAdapter.sol
Branch	main
Commit Hash	f2287ca3f395bc1eb15e46f673a94d9cd1e21a50
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th February 2025 - 5th February 2025
Updated Code Received	11th February 2025
Review 2	11th February 2025 - 18th February 2025
Fixed In	115460c69907938452f86ca84b9fb91a5bdf8122



Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0(0.0%)
High	0(0.0%)
Medium	0(0.0%)
Low	0(0.0%)
Informational	3 (100.0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	3



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Lack of Input Validation	Informational	Resolved
2	Floating Pragma	Informational	Resolved
3	Use Two-Step Ownership Transfer Instead of Standard Ownable	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



 **Missing Zero Address Validation**

 **Private modifier**

 **Revert/require functions**

 **Multiple Sends**

 **Using suicide**

 **Using delegatecall**

 **Upgradeable safety**

 **Using throw**

 **Using inline assembly**

 **Style guide violation**

 **Unsafe type inference**

 **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Informational Issues

Lack of Input Validation

Resolved

Path

contracts/MyOFT.sol
contracts/PPOFTAdapter.sol

Function Name

`constructor(...)` in `MyOFT`
`constructor(...)` in `PPOFTAdapter`

Description

The constructors of the `MyOFT` and `PPOFTAdapter` contracts do not verify whether critical input addresses are non-zero.

In `MyOFT`, the `\_lzEndpoint` and `\_delegate` addresses are passed directly to the `OFT` and `Ownable` constructors without explicit zero-address validation.

Similarly, in `PPOFTAdapter`, the `\_token`, `\_lzEndpoint`, and `\_delegate` addresses are not validated before being forwarded to the `OFTAdapter` and `Ownable` constructors.

While some LayerZero base contracts may perform internal checks, relying on upstream validation reduces clarity and increases the risk of misconfiguration during deployment. Initializing any of these addresses to `address(0)` could result in loss of ownership, broken cross-chain functionality, or a non-functional adapter.

Recommendation

Add explicit zero-address validation for all critical constructor parameters before passing them to parent constructors.



Floating Pragma

Resolved

Path

oft/contracts/MyOFT.sol
oft/contracts/PPOFTAdapter.sol

Description

Floating pragmas allow the contract to be compiled with multiple compiler versions. This can lead to unexpected behavior, as different compiler versions may introduce breaking changes, bug fixes, or different optimizations.

Impact - Low

Using a floating pragma can result in:

- Inconsistent bytecode across deployments
- Potential incompatibility with future compiler versions
- Increased risk during audits and production deployments

Likelihood

Low

Recommendation

Use a fixed Solidity compiler version instead of a floating pragma.



Use Two-Step Ownership Transfer Instead of Standard Ownable

Resolved

Path

oft/contracts/MyOFT.sol
oft/contracts/PPOFTAdapter.sol

Description

Both contracts inherit from OpenZeppelin's Ownable contract which implements a single-step ownership transfer mechanism. This design pattern is dangerous because if the owner accidentally transfers ownership to an incorrect address (due to a typo, wrong address copied, or any other human error), the contract becomes permanently inaccessible. There is no way to recover from this mistake, and all owner-restricted functions become permanently locked.

Impact

If ownership is transferred to an incorrect address (typo, wrong network address, non-existent address, or address without private key access), all admin functions become permanently inaccessible

Likelihood

Low

Recommendation

Implement Two-Step Ownership Transfer using OpenZeppelin's Ownable2Step contract instead of Ownable



Functional Tests

Some of the tests performed are mentioned below:

PPOFTAdapter.sol

- ✓ Constructor correctly initializes the OFTAdapter with the underlying ERC20 token address, LayerZero endpoint, and delegate address while setting up ownership.
- ✓ Cross-chain send operations properly lock tokens in the adapter contract and emit transfer events with accurate destination chain and recipient details.
- ✓ Cross-chain receive operations unlock the correct amount of tokens from the adapter and transfer them to the intended recipient address on the origin chain.

MyOFT.sol

- ✓ Constructor correctly deploys a new OFT token with specified name, symbol, and LayerZero endpoint while establishing initial ownership and delegate configuration.
- ✓ Cross-chain send operations burn tokens from the sender on the source chain and emit MessageSent events with accurate payload and destination details.
- ✓ Cross-chain receive operations mint the equivalent amount of tokens to the recipient on the destination chain based on verified LayerZero messages.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
OFTAdapter.sol	LayerZero Base Contract	Token locking/unlocking for cross-chain transfers	Correct custody and message handling implementation	Token loss, unauthorized unlocking, message replay
OFT.sol	LayerZero Base Contract	Token locking/unlocking for cross-chain transfers	Proper supply tracking across chains	Supply inflation, cross-chain desync
Ownable.sol	OpenZeppelin Access Control	Ownership management	Single owner has complete control	Single point of failure, irreversible ownership loss
LayerZero Endpoint	External Protocol	Cross-chain message verification and routing	Messages are authentic and tamper-proof	Message interception, replay attacks, endpoint compromise
Solidity 0.8.22	Compiler	Contract compilation	No critical compiler bugs	Version-specific vulnerabilities, optimizer bugs



2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
PPOFTAdapter	constructor()	What this function can do Initialize adapter with token address, LayerZero endpoint, and delegate address What this function cannot/should not do Be called after deployment, allow zero addresses, reinitialize ownership Main invariant(s) Token and endpoint addresses are immutable after deployment Access level Public (one-time at deployment)
PPOFTAdapter	send()	What this function can do Lock user's ERC20 tokens in adapter and send cross-chain message via LayerZero What this function cannot/should not do Send tokens without user approval, bypass LayerZero verification, unlock on same chain Main invariant(s) Tokens locked in adapter equals tokens to be unlocked on destination chain Access level Public (any token holder with approval)

Contract Name	Function	Category
PPOFTAdapter	_credit()	What this function can do Unlock tokens from adapter to recipient based on verified LayerZero message What this function cannot/should not do Be called without valid LayerZero message, unlock more than adapter balance, credit wrong recipient Main invariant(s) Message nonce used only once, adapter balance decreases by exact amount Access level Internal (LayerZero endpoint only)
MyOFT	constructor()	What this function can do Deploy new OFT token with name, symbol, LayerZero endpoint, and delegate What this function cannot/should not do Be called after deployment, reinitialize base contracts, allow zero endpoint Main invariant(s) Token metadata and endpoint are immutable after deployment Access level Public (one-time at deployment)



Contract Name	Function	Category
MyOFT	send()	What this function can do Burn tokens from sender and send cross-chain message via LayerZero What this function cannot/should not do Send without sufficient balance, skip token burning, bypass message verification Main invariant(s) Tokens burned on source chain equals tokens minted on destination chain Access level Public (any token holder)
MyOFT	_credit()	What this function can do Mint tokens to recipient on destination chain based on LayerZero message What this function cannot/should not do Be called without valid message, mint to zero address, mint incorrect amounts Main invariant(s) Message processed only once, total supply increases by exact amount Access level Internal (LayerZero endpoint only)



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
PPOFTAdapter.sol	ERC20	Locked user tokens during cross-chain transfers from origin chain to destination chains
MyOFT.sol	Native OFT Token	No custody - tokens are burned on send and minted on receive (transient state only)



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
PPOFTAdapter.sol	send()	ERC20	—	Public	Insufficient approval, reentrancy risk, frontrunning, permanent lock on message failure
PPOFTAdapter.sol	_credit()	—	ERC20	Internal	Message replay, unauthorized unlock, insolvency, wrong recipient
MyOFT.sol	send()	OFT (burned)	—	Public	Burn without mint on destination, cross-chain supply desync
MyOFT.sol	_credit()	—	OFT (minted)	Internal	Unauthorized minting, message replay, supply inflation



Closing Summary

In this report, we have considered the security of PuffpawOFT. We performed our audit according to the procedure described above.

No critical issues in PuffpawOFT Token. Three issues of Informational severity were found, which have been resolved by the PuffpawOFT team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

February 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com